

helixqa_integration_sec05

5. Phase 4: Test Type & Challenge Coverage Matrix

The HelixQA integration must achieve “100% coverage of all supported test types and the Challenges” per the project constitution [1]. This chapter defines ten test types, maps each against five client application categories, establishes challenge design standards that prevent bluff tests from passing when features are broken, and sets quantitative coverage measurement criteria with exact tools and targets.

5.1 Test Type Definitions

Every test type must declare three properties: what it validates, what it explicitly does not validate, and the anti-bluff verification method that proves the test exercises real functionality rather than a proxy. Test type numbering follows the HelixQA test bank schema convention [1].

Table 1 — Test Type Definitions and Anti-Bluff Methods

Test Type	What It Tests	What It Does NOT Test	Anti-Bluff Verification Method
Unit	Individual Go functions in <code>*_test.go</code> with <code>go test -short</code> [1]	Cross-package interactions, I/O side effects, real network or database calls	Mutate function body (swap return, invert conditional); test MUST fail. Revert; MUST pass.
Integration	Real component interactions: database writes, HTTP round-trips, service calls [2]	End-user UI flows, visual rendering, third-party inference quality	Replace real dependency with broken stub (e.g., <code>ErrConnectionRefused</code>); test MUST fail. Restore; MUST pass.
E2E	Full user workflows: client action → API → database → client state update [2]	Internal algorithm correctness, single-function edge cases	Replace API endpoint with 404; pipeline MUST fail. Failure evidence must include error-state screenshot.
Functional	Feature-level correctness with real user actions and visible outcomes [2]	Resilience under failure, performance at scale, security boundaries	Introduce UI regression (hide button via <code>display: none</code>); screenshot or vision analysis MUST detect absence.
Security	Auth bypass, CSRF, XSS, SQL injection,	Cryptographic strength, physical hardware security, OS sandboxing	Attempt bypass with stripped headers / malformed payload; security check MUST reject.

Test Type	What It Tests	What It Does NOT Test	Anti-Bluff Verification Method
	permission escalation ^[1]		Disable check; test MUST detect hole.
Stress	High concurrency, large payloads, memory pressure, connection exhaustion ^[1]	Functional correctness under normal load, UI usability during stress	Saturate resource (e.g., 10 000 connections); verify graceful degradation. <code>pkg/detector</code> must report <code>HasCrash: false</code> ^[1] .
Chaos	Resilience to random failures: network partitions, container kills, DB restarts mid-session ^[2]	Deterministic failure recovery, known-bad input handling	Kill dependency during active test; screenshot must show degraded state (spinner, error banner), not blank screen.
Benchmark	Performance baselines: latency percentiles, throughput, allocation rates ^[1]	Real-world perceived performance, network latency variability	<code>benchstat</code> comparison against stored baseline; regression > 5% MUST fail. Deliberately degrade algorithm; benchmark MUST detect.
Challenge	Real-life scenarios in <code>challenges/scripts/</code> validating actual behavior, not exit codes ^[1]	Synthetic unit assertions, mocked component behavior	Break challenged feature; challenge MUST fail. Restore; MUST pass. Validate with protocol-layer probe plus screenshot.
Runtime Verification	Live invariants: health checks, circuit breaker state, probe responses ^[2]	Offline code paths, startup-only checks, build-time validations	Force circuit breaker open; probe MUST report <code>unhealthy</code> . Restore; MUST report <code>healthy</code> within breaker timeout.

The anti-bluff mandate follows CONST-035: “no simulated or test-only implementation may pass a test that claims to verify production behavior” ^[3]. For unit tests, mutation is applied at the source level and the suite is run with `go test -short -run TestX`. For integration and E2E tests, the break is injected via environment toggles or proxy stubs. For visual tests, the break is a CSS or layout change; the vision analyzer (SSIM or vision LLM) must detect the delta.

Tests requiring real databases, devices, or HTTP calls carry build constraints such as `// go:build integration` or `//go:build e2e`; pure unit tests run under `go test -short` with no tag ^[1].

5.1.1 Unit Tests

Unit tests verify individual functions in isolation. Mocks are permitted only within `*_test.go` files executed under `go test -short`. Mocks must be injected through interfaces such as `CommandRunner` in `pkg/detector` and must not leak into production code [1]. The anti-bluff check is the “deliberately break it” protocol: edit the function to return an incorrect result, run the test, confirm failure, revert, and confirm success.

5.1.2 Integration Tests

Integration tests exercise real databases, real HTTP calls, and real service interactions. No mocks are allowed. HelixCode declares `make test-integration` as the target, running against a Docker Compose stack (`docker-compose.test.yml`) with PostgreSQL, Redis, and the application server [3]. The anti-bluff check replaces a real dependency with a broken endpoint; the test must fail with a connection error. Restore the real DSN; the test must pass.

5.1.3 E2E Tests

E2E tests model full user workflows: client action → API request → database persistence → API response → client state update. The HelixQA E2E suite lives in `tests/e2e/` and runs via `make test-e2e` [3]. The autonomous pipeline captures a screenshot at every action and performs pre/post validation around each step [2]. The anti-bluff check replaces the API endpoint with a static 404 handler; the test must fail, and the failure evidence must include a screenshot of the error state.

5.1.4 Functional Tests

Functional tests isolate a single feature and verify only that feature’s surface. The HelixQA `TestCase` schema supports functional categorization via `category: functional` in bank YAML files [1]. The anti-bluff check uses visual regression: disable the upload handler, capture the resulting UI, and assert that the screenshot differs from baseline by $SSIM < 0.95$ [2].

5.1.5 Security Tests

Security tests validate authentication, authorization, and injection resistance. HelixCode provides `cmd/security_test/` and `make security-test` [3]. Tests must attempt concrete bypasses: CSRF token omission, XSS payload injection, SQL injection, and JWT header stripping. The anti-bluff check is two-sided: first confirm the system rejects the attack; then temporarily disable the security control and confirm the attack succeeds.

5.1.6 Stress Tests

Stress tests exercise the system under high concurrency, large payloads, memory pressure, and connection exhaustion. HelixQA banks include `edge-cases-stress.yml` and `performance-validation.yml` [1]. The anti-bluff check monitors `pkg/detector` during the run: `HasCrash` and `HasANR` must remain `false` [1]. If the stress test passes but the detector finds a crash, the test is a bluff.

5.1.7 Chaos Tests

Chaos tests introduce random failures via `tc qdisc`, `docker kill`, database restarts mid-session, and CPU throttling. HelixQA supports chaos through `PipelineConfig.CompetingAppPackages` and environment variables such as `HELIXQA_CAPTURE_ANDROID_STUB` [1][2]. The anti-bluff check requires a screenshot during the induced failure. `IsBlankScreenshot()` — sampling 81 points on a 9×9 grid with per-channel range threshold < 20 — rejects blank frames [2].

5.1.8 Benchmark Tests

Benchmark tests establish baselines via Go’s `func BenchmarkX(b *testing.B)` convention in `tests/benchmark/` [1]. The anti-bluff check uses `benchstat` to compare against a stored baseline. A deliberate degradation (e.g., `time.Sleep(10 * time.Millisecond)` in the hot path) must cause a statistically significant regression ($p < 0.05$). Regression > 5% fails the CI gate.

5.1.9 Challenge Tests

Challenge tests are real-life scenario scripts in `challenges/scripts/` [1]. HelixCode defines a 7-phase suite: Basic Functionality, Configuration Management, LLM Provider Integration, Agent Execution, Memory System, Security & Compliance, and Performance & Scaling [3]. The anti-bluff check is the most rigorous: break the challenged feature, run the challenge, confirm failure, restore, confirm success. Verification uses protocol-layer probes plus screenshot evidence.

5.1.10 Runtime Verification

Runtime verification tests live system invariants through periodic health checks and circuit breaker validation. The HelixQA vision server exposes `/health` for liveness and `/learning/stats` for cache-hit metrics [2]. The anti-bluff check forces a circuit breaker open (return 503 from downstream) and confirms the health endpoint reports `unhealthy`. When downstream recovers, the breaker must close and health must report `healthy` within the breaker timeout (typically 30 s).

5.2 Client App Coverage Matrix

HelixCode ships six client targets: Web, Desktop (Linux, macOS, Windows), Mobile (Android, iOS), CLI, and TUI [3]. Desktop is treated as a single category with OS-specific tooling noted per cell; Mobile is split into Android and iOS sub-rows where tooling differs. Table 2 presents 10 test types × 5 client categories = 50 cells, each specifying the exact tool, executor, and verification method.

Table 2 — Client App Coverage Matrix: Test Type × Client Category

Test Type	Web (Browser)	Desktop (Fyne GUI)	Mobile (Android / iOS)	CLI (Cobra)
Unit		<code>go test -short</code> on Fyne widgets;	<code>go test -short</code> on gomobile	<code>go test -short</code> on <code>cmd/cli/</code>

Test Type	Web (Browser)	Desktop (Fyne GUI)	Mobile (Android / iOS)	CLI (Cobra)
	<code>go test -short</code> on web/ <code>src/capture/</code> ; <code>jsdom</code> for frontend logic	<code>fyne test</code> headless app	bindings; Android: Robolectric; iOS: XCTest	and <code>cmd/</code> <code>root.go</code> ; Cobra command tree validation
Integration	Playwright + real backend API (<code>docker-compose.test.yml</code>); no mock HTTP client	X11/Playwright against running desktop binary; real API calls	ADB + real device/emulator; <code>adb shell am</code> <code>start</code> with real backend; iOS: <code>xcrun simctl</code> + real API	<code>os/exec</code> of compiled CLI binary against real server; <code>stdout/stderr</code> parsed with real string ops
E2E	Playwright full journey: login → create agent → run → verify; screenshots at every step	X11 executor: <code>launch</code> → navigate menus → trigger action → verify window title; <code>pkg/navigator/</code> <code>x11_executor.go</code> [¹]	ADBExecutor: <code>tap</code> → type → screenshot → vision verify; <code>pkg/</code> <code>navigator/</code> <code>executor.go</code> [¹]; iOS: <code>simctl</code> <code>io +</code> WebDriverAgent	CLI chain: <code>helix</code> <code>login</code> → <code>helix</code> <code>agent</code> <code>create</code> → <code>helix</code> <code>agent run</code> → parse output for success token
Functional	Playwright per-feature: upload, chat, settings; responsive breakpoints (375×667, 768×1024, 1920×1080) [²]	<code>Fyne</code> <code>test.NewApp()</code> headless widget validation; pixel-diff for OpenGL canvas visual regression	Feature isolation: camera permission, deep link, geo- restriction probe (<code>pkg/</code> <code>autonomous/</code> <code>geo_probe.go</code>) [¹]	Per-command <code>helix</code> <code>config set</code> → verify config file; <code>helix</code> <code>status</code> → verify exit code 0 + “Status: OK”
Security	Playwright + OWASP ZAP proxy; XSS in chat input; CSRF token check	X11 automation accessing admin menu without auth; verify login screen shown	ADB: intercept traffic with <code>tcpdump / Frida</code> bridge (<code>pkg/</code> <code>observe/</code> <code>frida</code>) [¹]; verify TLS pinning	CLI: <code>helix</code> <code>admin</code> without valid token; verify exit code ≠ 0 and <code>stderr</code> contains “unauthorized”
Stress	Playwright: 50 concurrent browser contexts hitting chat; p95 via <code>performance.mark</code>	X11: rapid window open/close; monitor X11Executor memory via <code>/proc/</code> <code>[pid]/status</code>	ADB: <code>adb</code> <code>shell monkey</code> 10 000 events; <code>pkg/detector</code> checks ANR (<code>HasANR:</code> <code>false</code>) [¹]	Parallel CLI: 100 concurrent <code>helix</code> <code>status</code> ; verify no panic in <code>stderr</code>

Test Type	Web (Browser)	Desktop (Fyne GUI)	Mobile (Android / iOS)	CLI (Cobra)
Chaos	Playwright + <code>tc_qdisc</code> on server; screenshot must show retry spinner or error, not blank	Kill desktop process mid-action; restart; verify recovery or graceful error dialog	<code>adb shell am force-stop</code> mid-transaction; restart; verify data persisted or error shown	<code>kill -9</code> CLI process mid-write; verify partial state does not corrupt config
Benchmark	Playwright <code>performance.mark</code> → measure for page load; Lighthouse CI for Core Web Vitals	<code>go test -bench</code> on Fyne render loop; frame time via OpenGL query counters	ADB <code>dumpsys gfxinfo</code> for frame timing; <code>scrpy</code> latency measurement	<code>go test -bench</code> on CLI dispatch; <code>perf</code> for syscall overhead
Challenge	7-phase challenge via <code>challenges/scripts/run_all_challenges.sh</code> [3]; Phase 3 with real inference call	Desktop challenge: full agent creation and execution; window state verified with screenshot diff	Mobile challenge: onboard → login → create task → verify push notification; deep link validation	CLI challenge: <code>helix run</code> from empty project to generated code; verify output directory contents
Runtime Verification	Periodic <code>/health</code> probe; vision server <code>/learning/stats</code> cache hit rate > 0.8 [2]	Process alive via <code>pidof</code> ; window responsive to <code>xte</code> key event within 2 s	ADB <code>pidof + dumpsys activity</code> probe every 5 s; ANR detection [1]	<code>helix version</code> as liveness probe; exit code 0 + semantic version regex

No client category is exempt from any test type. CLI and TUI use terminal-state capture (`asciinema`, `tmux capture-pane`, `tc` event injection) as screenshot-equivalent evidence [2]. Web testing uses `pkg/navigator/playwright_executor.go` (~8.2 KB) [1]. Desktop Linux uses `pkg/navigator/x11_executor.go` (~3.5 KB) with `xte` and `import -window root png:-` [1][2]. Android uses `pkg/navigator/executor.go` (~14 KB) for tap, type, screencap, and swipe [1].

Web testing (5.2.2) uses Playwright with the bridge script `scripts/playwright-bridge.js` [1][2]. Visual regression in `pkg/regression/visual.go` applies SSIM threshold 0.95 or perceptual hashing [2]. Responsive breakpoints at 375×667, 768×1024, and 1920×1080 use `CaptureResponsive()` [2]. Accessibility integrates `axe-core` scanning for WCAG 2.1 Level AA violations.

Desktop (5.2.3) uses Fyne v2 [3]. `X11Executor` handles screenshot via `import -window root` and input via `xte` [1]. Multi-monitor capture currently captures root window only; the proposed `LinuxEngine` adds per-display enumeration via `xrandr` or `wl_output` [2]. Crash detection monitors PID and `stderr` via `pkg/detector` [1].

Android (5.2.4) uses ADB through ADBExecutor [1]. pkg/detector polls logcat for FATAL EXCEPTION and ANR in plus pidof [1]. pkg/autonomous/geo_probe.go curls endpoints and marks GEO_RESTRICTED for skipped tests [1]. Deep link validation uses adb shell am start -W and dumsys activity.

iOS (5.2.5) is absent from current capture [2]. The integration adds iOSSimulatorEngine via xcrun simctl io screenshot and recordVideo [2]. WebDriverAgent provides real-device automation. SSIM 0.95 applies to screenshot diff [2]. Lifecycle testing exercises simctl launch --terminate-running and foregrounding.

CLI (5.2.6) uses Cobra and Viper [3]. Tests invoke the compiled binary via os/exec, parsing stdout/stderr and verifying exit codes [2]. CLIExecutor returns stdout as its “screenshot”; the proposed CLIEngine adds rendered mode using asciinema [2]. ANSI capture verifies escape sequences appear when --color is set.

TUI (5.2.7) uses rivo/tview and gdamore/tcell/v2 [3]. The TUIEngine supports asciinema recording, tmux capture-pane, and xterm.js in a headless browser [2]. Keyboard simulation injects tcell EventKey values. Screen buffer comparison reconstructs the terminal grid from ANSI sequences. Color verification checks tcell.Style flags.

5.3 Challenge Design Standards

Challenges are the highest-fidelity tests. A script that merely checks exit codes or greps for static strings is a bluff.

Table 3 — Challenge Design Template

Phase	Action	Verification Method	Anti-Bluff Check
Setup	Start dependencies (Docker Compose, emulator, server), initialize data, verify pre-conditions	Protocol-layer probe: TCP connect + HTTP GET /health → 200 + JSON status: "up"	Temporarily break health endpoint (return 503); setup MUST fail. Restore; MUST pass.
Execution	Run the user workflow: tap/type/click/navigate through real UI or send real API requests	Screenshot before + after action; IsBlankScreenshot() must return false for both [2]	Hide target button/field; screenshot diff (SSIM < 0.95) or vision LLM must detect missing element.
Verification	Assert goal state reached: order	Protocol-layer probe: real HTTP request with real payload; parse	Mutate API response schema (remove required

Phase	Action	Verification Method	Anti-Bluff Check
	confirmed, file uploaded, chat response received	real JSON response; do NOT grep for “success”	field); verification MUST fail. Restore; MUST pass.
Teardown	Stop dependencies, clean data, reset environment to known state	Post-teardown probe: GET / health returns down or connection refused; database tables empty	Skip teardown; subsequent test MUST detect stale data or conflicting state.

Every Challenge must traverse all four phases. The autonomous 4-phase lifecycle in `pkg/autonomous/phase.go` defines setup, doc-driven, curiosity, and report phases ^[1]. Challenge scripts follow setup, execution, verification, teardown. Setup must validate every dependency before execution. Teardown must run even if execution or verification fails; this is enforced by a `defer` block in the Challenge runner. The “floor” of shallow connectivity (`curl -I` returning 200) is rejected in favor of the “ceiling”: a real request with a real payload producing a real response whose fields are type-checked ^[2].

For HTTP APIs, the floor is `curl -I` returning 200; the ceiling is a `POST` with real JSON payload, receiving a response whose body is parsed and type-checked. For ADB, the floor is `adb devices` listing a device; the ceiling is `adb shell screencap -p` returning valid PNG bytes (> 5 000 bytes, decodable, not blank) ^[2]. For WebSocket, the floor is `ws.Dial()` succeeding; the ceiling is real message exchange with screenshot verification of the resulting UI state.

Visual Challenges capture screenshots immediately before and after the user action. Images are compared using SSIM with threshold 0.95, stored in `HELIX_VISION_SSIM_THRESHOLD` ^[1] ^[2]. If SSIM exceeds the threshold, the screen did not change and the action may have failed silently. Alternatively, a vision LLM (Qwen2.5-VL, GLM-4V, or Ollama `minicpm-v:8b`) answers a structured question such as “Does the confirmation page show an order number?” ^[2]. The vision server /analyze endpoint accepts base64 PNG + prompt and returns `VisionResult` ^[2].

For every Challenge, a companion “break test” runs on a separate branch or with an environment flag:

```
# Break test: branch where login handler returns 401 for all
#               credentials
HELIX_ANTIBLUFF_BREAK=1 go test ./challenges/... -run
#               TestChallengeLoginFlow
# Expected: FAIL

# Normal test: main branch with correct handler
HELIX_ANTIBLUFF_BREAK=0 go test ./challenges/... -run
#               TestChallengeLoginFlow
# Expected: PASS
```


If the Challenge passes when `HELIX_ANTIBLUFF_BREAK=1`, it is a bluff. The break test result is recorded in the test report alongside the normal result.

5.4 Coverage Measurement

Coverage is measured across four dimensions: code, feature, platform, and provider.

Table 4 — Coverage Measurement Criteria

Metric	Target	Measurement Method	Tool
Code coverage — critical paths	100%	<code>go test -coverprofile=coverage.out; go tool cover -func</code>	Go built-in cover; <code>golangci-lint</code> with <code>gocov</code> reporter
Code coverage — all packages	$\geq 80\%$	Same as above, aggregated across <code>pkg/</code> *	<code>make test-cover</code> [3]
Feature coverage	100%	Map every <code>TestCase</code> in <code>banks/</code> to ≥ 1 Challenge and ≥ 1 E2E test	<code>pkg/testbank/manager.go</code> filtering by category and platform [1]
Platform coverage	Every client on every supported OS	Execute full suite on Linux, macOS, Windows, Aurora OS, Harmony OS	Docker Compose for Linux; GitHub Actions runners for macOS/Windows; emulator for Aurora/Harmony [3]
Provider coverage	Every LLM provider with ≥ 1 real inference call	<code>LiveChat()</code> or <code>Vision()</code> per provider per release; verify structurally valid response	<code>pkg/llm</code> adaptive provider with health probe; <code>internal/verifier/</code> for model scoring [1] [3]
Anti-bluff coverage	100% of non-unit tests	Every integration, E2E, functional, and Challenge test has a deliberate-break companion	<code>scripts/anti-bluff-verify.sh</code> orchestrates break/restore cycles [2]
Screenshot coverage	100% of UI tests produce ≥ 1 screenshot	<code>pkg/session/recorder.go</code> indexes screenshots; verify <code>screenshotIdx > 0</code>	<code>SessionRecorder</code> timeline JSON [1]
Crash/ANR coverage	0 crashes, 0 ANRs per run	<code>pkg/detector</code> <code>HasCrash</code> and <code>HasANR</code> must be <code>false</code>	Detector output in <code>timeline.json</code> [1]

The Go toolchain provides `go test -coverprofile=coverage.out`, consumed by `go tool cover -func` for per-function percentages. HelixCode declares `make test-cover` as the target [3]. Critical paths — `internal/auth/`, `internal/llm/`, `internal/server/`, and `pkg/llm/` — target 100% line coverage. Non-critical packages relax to $\geq 80\%$. Coverage is not a substitute for anti-bluff validation: 100% coverage with all mocks is a bluff.

Feature coverage is tracked through `TestCase` in `banks/` files. Each `TestCase` has a `category` field and a `platforms` array [1]. For every user-facing feature in product documentation, there must exist at least one `TestCase` in a Challenge bank and one in an E2E bank. `pkg/testbank/manager.go` filtering generates a gap report: features with zero matching `TestCase` records are flagged as untested [1].

HelixCode supports Linux, macOS, Windows, Aurora OS, and Harmony OS [3]. Every client application executes on every supported OS at least once per release. The Web client is OS-agnostic and tested via Playwright on all three major desktop OSes plus Dockerized Linux. The Desktop client (Fyne) is compiled per OS and tested via X11 on Linux, screenshot on macOS, and `PrintWindow` on Windows [2][3]. Aurora and Harmony use the same Fyne framework; testing is limited to build verification and smoke tests due to emulator availability [3]. Mobile is tested on Android emulator (all hosts) and iOS simulator (macOS host only). CLI and TUI compile and execute on all five OSes.

HelixCode integrates 14 LLM providers: OpenAI, Anthropic, Google Gemini, Ollama, LlamaCPP, Qwen, XAI, OpenRouter, GitHub Copilot, Azure OpenAI, AWS Bedrock, Vertex AI, KoboldAI, and a generic local provider [3]. Every provider receives at least one real inference call per release. The call must exercise `Chat()` or `Vision()`, not just `Health()`. Verification is structural: the response must be a valid `Response` with non-empty `Text` and a `Latency` field in milliseconds. `pkg/llm/adaptive.go` probes all configured providers based on health, latency, and cost [1]; this infrastructure is reused for coverage validation. `internal/verifier/` caches `VerifiedModel` structs with capability flags (`SupportsVision`, `SupportsStreaming`, `SupportsTools`) [3]; provider tests verify that advertised capabilities match actual behavior.

Example Challenge Script Template

The following Bash template implements the 4-phase Challenge standard for an “Agent Creation” workflow. It includes protocol-layer probes, screenshot verification, and deliberate-break hooks.

```
#!/usr/bin/env bash
# challenges/scripts/agent_creation_challenge.sh
# Phase 4 Challenge — Agent Creation Workflow
# Requires: Helix server on localhost:8080, Playwright or CLI
#           executor wired

set -euo pipefail

API_BASE="http://localhost:8080"
OUTPUT_DIR="${HELIX_OUTPUT_DIR:-./qa-results/challenges}"
mkdir -p "$OUTPUT_DIR"
```

— Phase 1: Setup

```
echo "[SETUP] Probing server health..."
HEALTH=$(curl -sf "$API_BASE/api/v1/health" || echo "{}")
if [[ $(echo "$HEALTH" | jq -r '.status // "down"') != "up" ]];
then
    echo "[FAIL] Setup: server not healthy"
    exit 1
fi

# Anti-bluff: HELIX_ANTIBLUFF_BREAK=1 causes server to return 503
if [[ "${HELIX_ANTIBLUFF_BREAK:-0}" == "1" ]]; then
    echo "[ANTI-BLUFF] Simulating broken health endpoint"
fi
```

— Phase 2: Execution

```
echo "[EXEC] Creating agent via API..."
RESPONSE=$(curl -sf -X POST "$API_BASE/api/v1/agents" \
    -H "Content-Type: application/json" \
    -d '{"name":"challenge-agent","model":"gpt-4o"}')
AGENT_ID=$(echo "$RESPONSE" | jq -r '.id')

# Screenshot capture (Web executor)
if command -v npx &> /dev/null; then
    npx playwright screenshot --viewport-size=1280,720 \
        "$API_BASE/agents/$AGENT_ID" "$OUTPUT_DIR/
        agent_created.png"
    if [[ $(stat -c%s "$OUTPUT_DIR/agent_created.png") -lt
        5000 ]]; then
        echo "[FAIL] Screenshot too small – likely blank"
        exit 1
    fi
fi
```

— Phase 3: Verification

```
echo "[VERIFY] Fetching agent and asserting fields..."
VERIFY=$(curl -sf "$API_BASE/api/v1/agents/$AGENT_ID")
if [[ $(echo "$VERIFY" | jq -r '.name') != "challenge-agent" ]];
then
    echo "[FAIL] Verification: agent name mismatch"
    exit 1
fi
if [[ $(echo "$VERIFY" | jq -r '.model') != "gpt-4o" ]]; then
    echo "[FAIL] Verification: agent model mismatch"
    exit 1
fi
```

```

# Anti-bluff: break-test branch removes required fields
if [[ "${HELIX_ANTIBLUFF_BREAK:-0}" == "1" ]]; then
    if echo "$VERIFY" | jq -e '.name' &>/dev/null; then
        echo
        "[FAIL] Anti-bluff: test passed when field should be
        missing"
        exit 1
    fi
fi

# — Phase 4: Teardown


---


echo "[TEARDOWN] Deleting test agent..."
curl -sf -X DELETE "$API_BASE/api/v1/agents/$AGENT_ID" || true

if curl -sf "$API_BASE/api/v1/agents/$AGENT_ID" &>/dev/null; then
    echo "[FAIL] Teardown: agent still exists after deletion"
    exit 1
fi

echo "[PASS] Agent Creation Challenge completed"

```

The script uses `set -euo pipefail` for strict error handling, `curl -f` for HTTP-level failure propagation, and `jq` for structured JSON parsing rather than string `grep`. The `HELIX_ANTIBLUFF_BREAK` environment variable provides the deliberate-break hook. Screenshot validation checks file size against the 5 000-byte minimum from `ADBExecutor` validation in `pkg/navigator/executor.go` ^[1]_[2]. Every Challenge script in the integration suite follows this template, adapting endpoint URLs, payloads, and executor commands to the feature under test.